

DSAA 4011 · LAB 5 · SATURDAY 10 OCTOBER 2026

Learning What We Never See

Expectation–Maximization at the Frontier

Lecture 5's **expectation–maximization** (EM): **guess** what is hidden, **count**, repeat. Today: five places where this simple loop runs modern AI.

Yuwen Huang

Advanced Machine Learning and Deep Learning · HKUST (Guangzhou)

Builds on Lecture 5: Learning what we never observe

Outline

- ▶ **Learning what we never see (Lecture 5)**

Troubles 1–2: cleaning and searching data

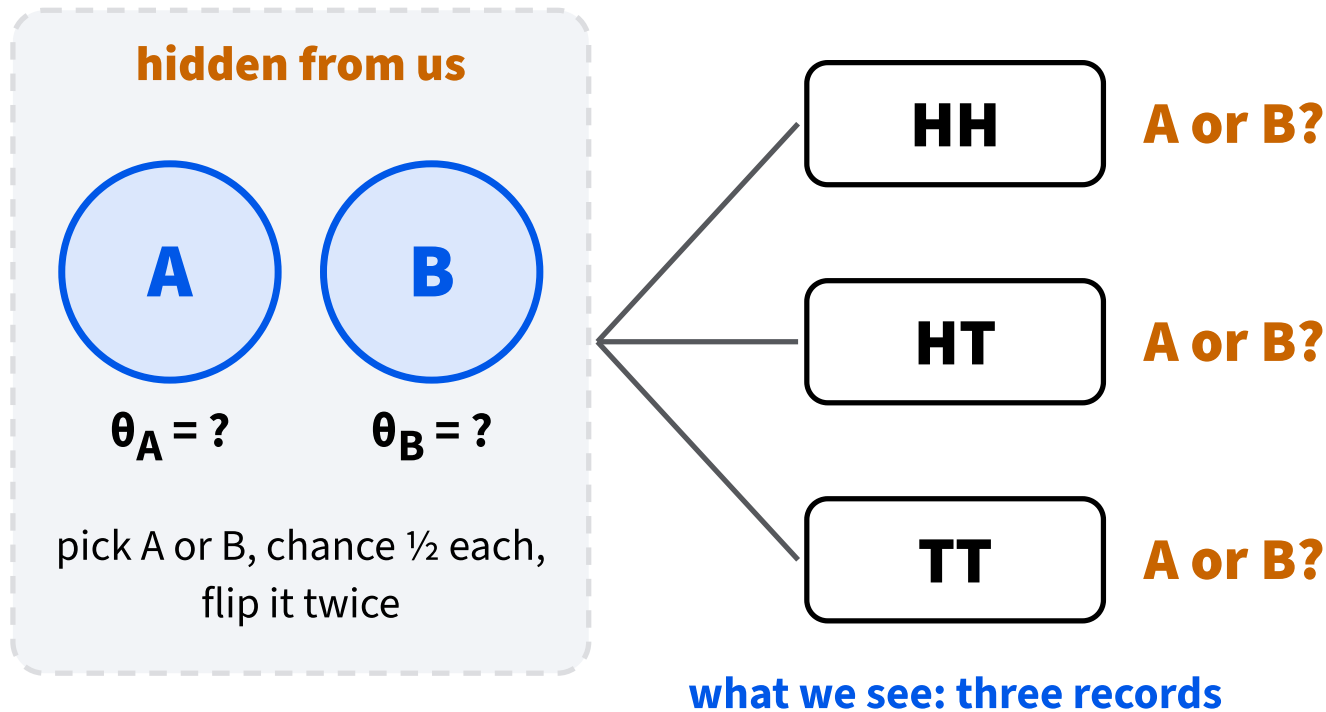
Troubles 3–5: inside image and language models

Conclusion and your turn

One loop, five troubles, five small fixes.

Learning what we never see

Two coins, one hidden: which coin made each record?



Look: we see the flips, **never the coin**.

Q: Can we learn θ_A and θ_B anyway?

A: Yes: guess the coin **softly**, count heads, repeat.

Learning what we never see: in three sentences

BACKGROUND Data hide part of the story: which coin, which label, which reasoning.

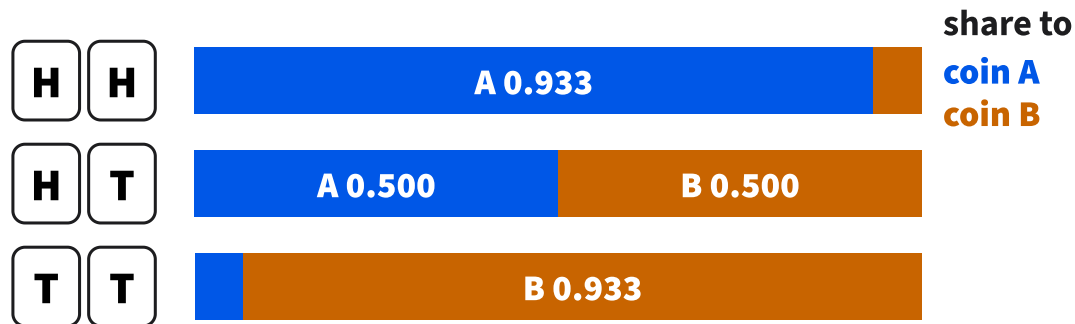
KEY IDEA **Guess** the hidden part softly (**E-step**), then **count** (**M-step**). Repeat.

USED FOR Every model with a hidden part [Dempster, Laird and Rubin, 1977].

Learning what we never see

Guess **softly**, **count**, repeat.

round 15 · M-step: count



heads and flips, credited by the soft labels:

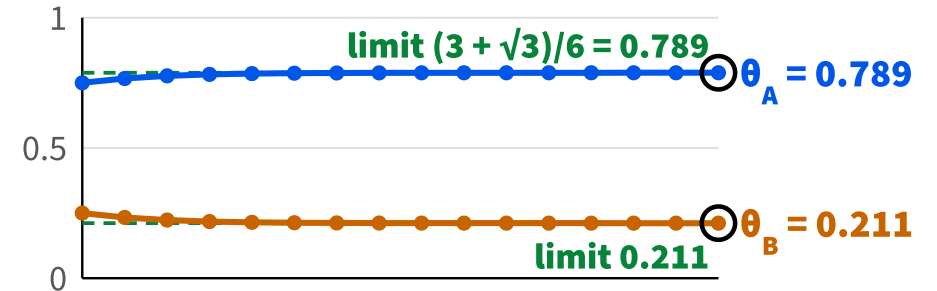
heads to A: $2 \cdot 0.933 + 1 \cdot 0.500 + 0 \cdot 0.067 = 2.366$

flips to A: $2 \cdot (0.933 + 0.500 + 0.067) = 3.000$

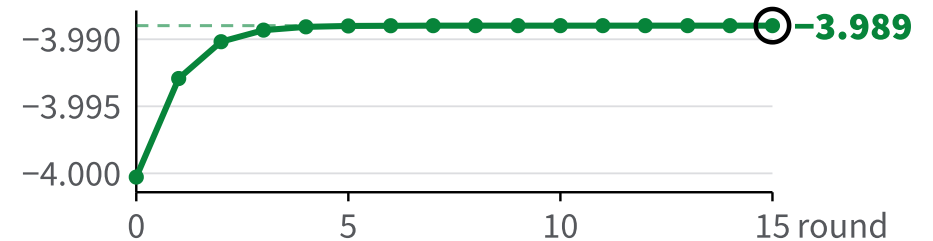
$\theta_A = 2.366 \div 3.000 = 0.789$

$\theta_B = 0.634 \div 3.000 = 0.211$

heads chances



log-likelihood $\ln p(\text{HH}, \text{HT}, \text{TT})$: never goes down



Watch: HH looks like coin A: **soft label** 0.9 (p. 46). Round by round θ_A climbs to **0.789**, and the fit (likelihood) **never drops**.

Q: Why not pick one coin per record? **A:** HT is 50 : 50; a hard pick is far too sure.

Try: another start.

Main Question

**EM only guesses and counts.
Can it judge a crowd, search a library, read images, cut
words, and teach reasoning?**

Yes: one small change each.

Learning what we never see

Five words for today.

Word	Plain meaning	Lecture 5 name
record x	what we see	observed data
hidden coin z	what we never see	hidden variable
soft label	how much we believe each z	posterior, E-step
count	heads weighted by soft labels	M-step
likelihood	chance of all records; never drops	Theorem 4

Learning what we never see

Five troubles at the frontier, five fixes.

	Trouble	Fix	Used for
1	people disagree on labels	???	clean training labels
2	too much data	???	search for chatbots
3	the hidden part is many numbers	???	image models
4	nobody marks where to cut words	???	tokenizers
5	the reasoning is never written	???	models that reason

Read: the fixes appear one by one.

Outline

Learning what we never see (Lecture 5)

- ▶ **Troubles 1–2: cleaning and searching data**

Troubles 3–5: inside image and language models

Conclusion and your turn

First: clean the labels, then search the data.

Trouble 1 in three sentences

- BACKGROUND** An AI learns from photos tagged “cat” or “dog” by people. Wrong tags teach it wrong.
- KEY IDEA** Find the **true tags** and **whom to trust** together: each one helps find the other.
- USED FOR** Cleaning crowd-sourced labels before training.

Trouble 1: people disagree

Trouble: three people tag the same photos, and **disagree**.

	photo 1	photo 2
person 1	cat	dog
person 2	cat	dog
person 3	dog	dog
true label	?	?

1. Photo 1: two say **cat**, one says **dog**.

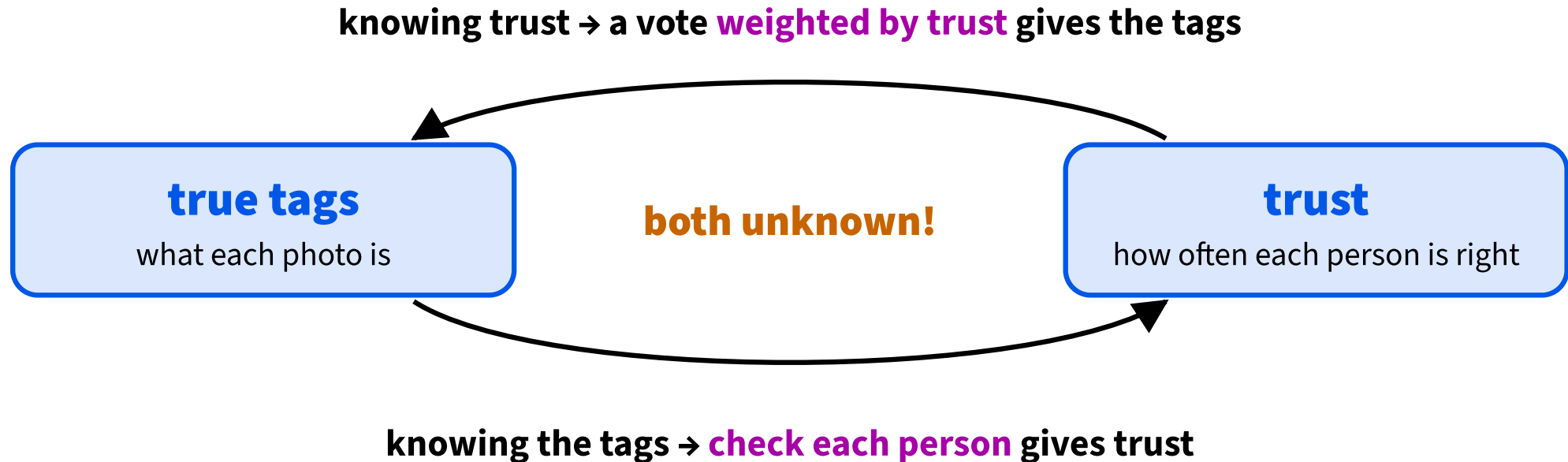
2. A vote says cat, but it trusts **everyone** **equally**.

Q: What if person 3 is careless?

A: Then their vote should count less. But who is careless?

Trouble 1: people disagree

Trouble: a chicken-and-egg problem.



Read: each unknown would give the other, but we have **neither**. Same as Lecture 5: the true tag is the hidden coin; trust is θ .

Q: Where do we start? A: Guess trust, then go round the loop: that is EM.

Trouble 1: people disagree

Fix: go round the loop. One round by hand:

START Trust everyone the same: **0.8**.

① **TAGS** Photo 1: cat, cat, dog → **80% cat**. Two votes beat one (**p. 48**).

② **TRUST** Person 3 went against the others on 2 of 6 photos → trust **0.8** → **0.72**.

REPEAT Person 3's vote now counts less, so the tags get surer, and person 3's trust settles at **0.67**.

Read: nobody told EM who is careless; it found out from the **disagreements**.

Trouble 1: people disagree

Result: after a few rounds, EM knows **whom to trust**.

round 8 • ② score each person

	person 1	person 2	person 3	what is the photo?
photo 1	cat	cat	dog	cat 1.000
photo 2	dog	dog	dog	dog 1.000
photo 3	cat	cat	cat	cat 1.000
photo 4	dog	dog	cat	dog 1.000
photo 5	cat	cat	cat	cat 1.000
photo 6	dog	dog	dog	dog 1.000

person 3 matches the guesses on photos 1 ... 6:

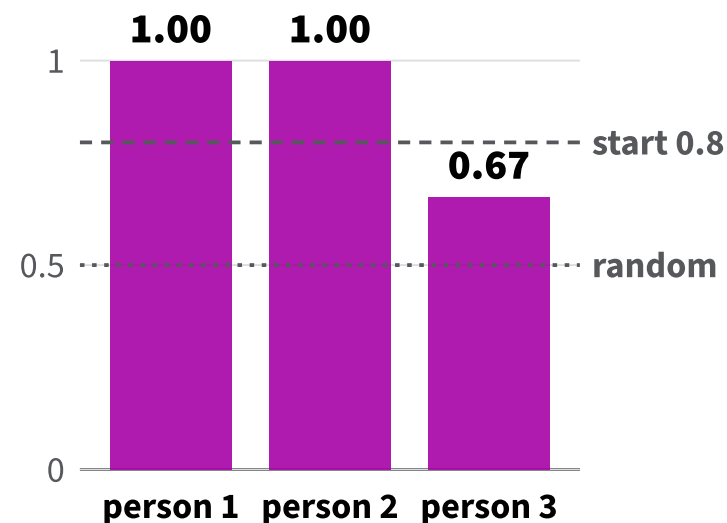
$$(0.000 + 1.000 + 1.000 + 0.000 + 1.000 + 1.000) \div 6 = 0.667$$

Watch: each round does ① then ②. The photos become sure (blue), person 3's trust falls to **0.67**, persons 1 and 2 rise to 1.

Q: Why does person 3 lose trust? A: They disagree with two people who agree with each other.

Try: person 3 guesses at random: trust 0.5, their tags stop counting. This is **Dawid-Skene EM** [Dawid and Skene, 1979].

trust: how often each person is right



persons 1 and 2 always agree:
EM ends up trusting them fully

Trouble 2 in three sentences

BACKGROUND

Before answering, a chatbot looks up texts in a library of millions. Each text is stored as a **point**: close points, similar meaning.

KEY IDEA

Sort the texts onto **shelves** once, with EM's hard-label version (**k-means**); each question then searches **one shelf**.

USED FOR

Vector search behind chatbots [Jégou, Douze and Schmid, 2011; Lewis et al., 2020].

Trouble 2: too much data

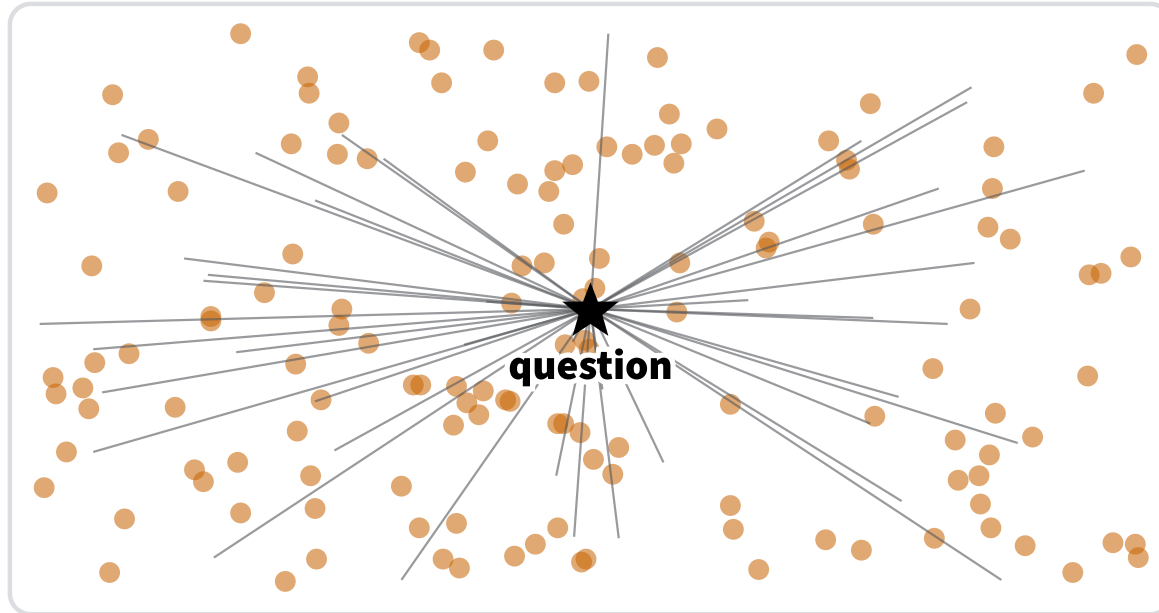
Scenario: the same EM as Lecture 5's numbers on a line.

	Lecture 5, Section 5	Chatbot search
record	a number: 0, 1, 4, 5	a text, stored as a point
hidden coin	which bell made it	which shelf (topic) it is on
θ	the bell centres	the shelf centres
E-step	a share for each centre	nearest centre only (hard)
M-step	weighted average	plain average

Read: Lecture 5, Theorem 5: shrink every bell to width zero, and EM becomes exactly the right column: **k-means**.

Trouble 2: too much data

Trouble: each question must find its **nearest** text.



every dot is a text: compare with all of them?

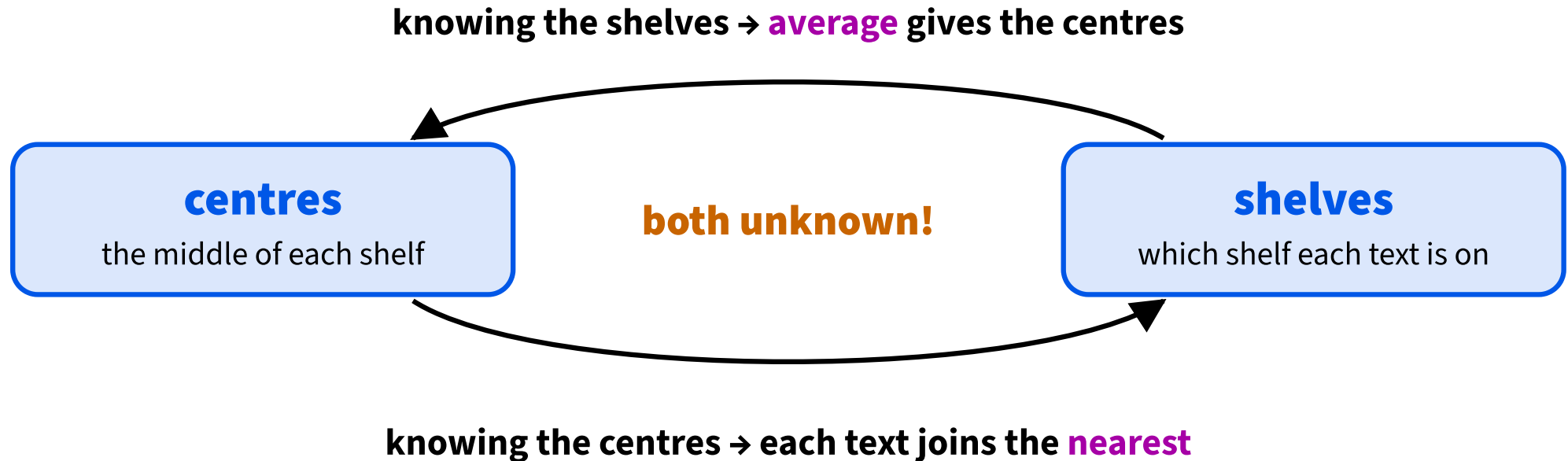
1. A question becomes a point too.
2. Plain search compares it with **every** text.
3. A million texts, thousands of questions a second: **too slow**.

Q: What would a librarian do?

A: Shelf books by topic, then search one shelf.

Trouble 2: too much data

Fix: shelf texts by topic. But that is **chicken and egg**.



Read: the same loop as EM. Start from guessed centres and go round: that is **k-means**.

Q: Why hard labels, not EM's soft shares? A: A text must sit on one shelf; “0.7 here, 0.3 there” cannot be filed.

Trouble 2: too much data

Fix: go round the loop. One round by hand:

- START** Six texts at 1, 2, 3, 10, 11, 12. Guess centres **2** and **3** (a bad guess).
- ① **E-STEP** Each text joins the **nearer** centre: {1, 2} and {3, 10, 11, 12}.
- ② **M-STEP** Each centre moves to its texts' **average**: **1.5** and **9**.
- REPEAT** Text 3 switches shelf; centres **2** and **11**; nothing moves: done.
- SEARCH** A question at 10.4: 2 centres → shelf {10, 11, 12} → **5 distances, not 6**.

Trouble 2: too much data

Why it works: fast, usually right, and a known price.

FAST

A question checks K centres + one shelf of about N/K texts. A million texts, 1,000 shelves: 2,000 distances, not 1,000,000: **500 times fewer**.

RIGHT

k-means never increases the total squared distance from texts to their centres (Lecture 5, score J), so shelves are **tight**: the nearest text is usually on the question's shelf.

PRICE

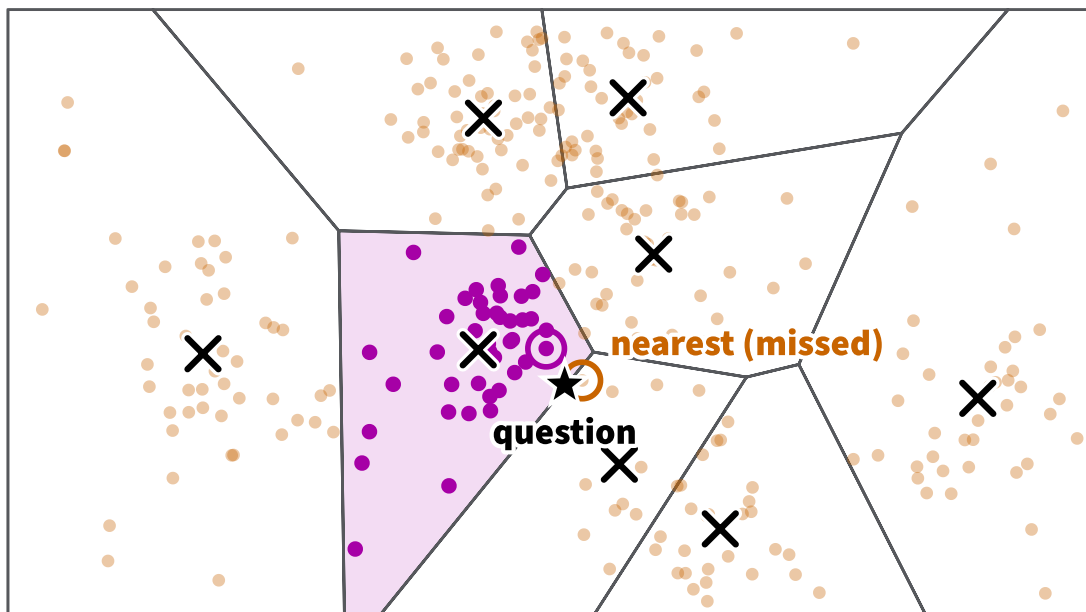
A neighbour just across a border is **missed**. Remedy: search the 2 or 3 nearest shelves.

Read: a little accuracy for a lot of speed: this is how vector databases search [Jégou, Douze and Schmid, 2011].

Trouble 2: too much data

Result: search **one shelf** (cell): 49 distances instead of 300.

texts as vectors (orange) · centres (black x)



search one cell (8 cells)

every text: 300

one cell: 8 + 41 = 49

nearest text: missed x

magenta ring: best in the cell, not the nearest

over 50 questions: 4% of nearest texts missed, about 50 distance computations each, not 300

Watch: the centres settle, then the question searches only its own cell. Sometimes the nearest text sits just across a border: **missed** (4%).

Q: Why are misses rare? A: Shelves are tight: the nearest text is usually near the same centre.

Try: more cells: faster, but more **misses**. This is **k-means search** [Lloyd, 1982].

Outline

Learning what we never see (Lecture 5)

Troubles 1–2: cleaning and searching data

► **Troubles 3–5: inside image and language models**

Conclusion and your turn

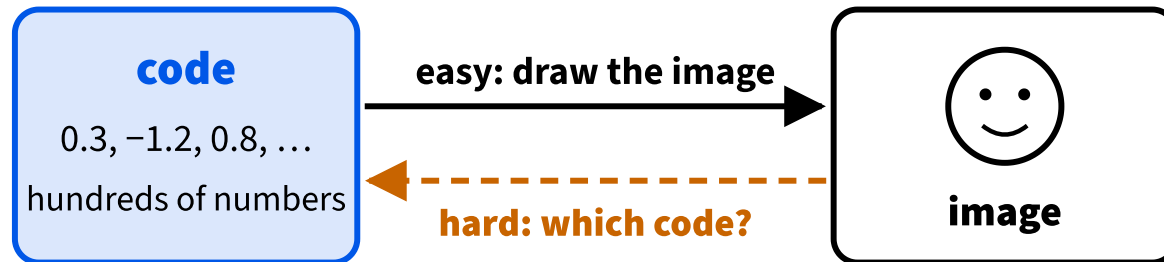
Next: inside image and language models.

Trouble 3 in three sentences

- BACKGROUND** An image model draws each image from a hidden **code**: hundreds of numbers (pose, colour, ...).
- KEY IDEA** Train a second network, the **encoder**, to guess each image's code in one pass.
- USED FOR** Image models that compress, generate and edit.

Trouble 3: the code is many numbers

Trouble: to learn, the model must know each image's **code**.



endless codes could draw it

1. EM's guess step asks: for this image, how likely is each code?

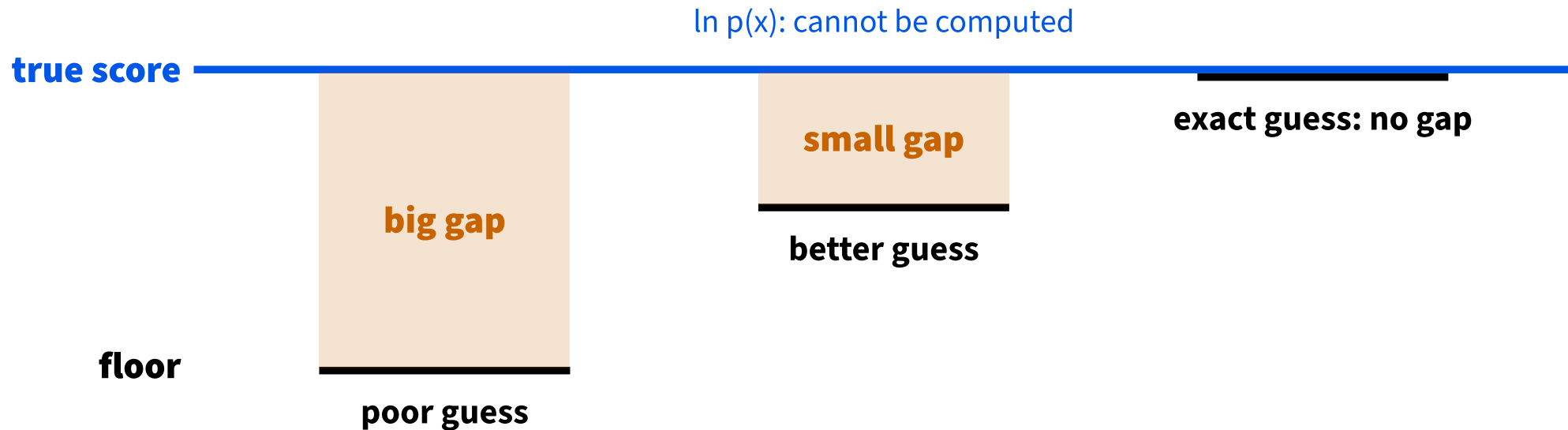
2. Two coins: 2 options. Codes: **endless** options, no formula.

Q: Why not try many codes?

A: Hundreds of numbers: far too many combinations.

Trouble 3: the code is many numbers

Trouble: the exact answer is out of reach.



Read: we cannot compute the true score, but **every guess gives a floor** under it. The better the guess, the smaller the **gap**.

Q: So what do we train? A: Raise the floor: it trains the model and the guesser together.

Trouble 3: the code is many numbers

Fix: go round the loop. One round by hand:

START A coin stands in for an image. On HH the exact soft label is **0.9**; the guesser says 0.5: gap **0.51**.

① **TRAIN** The guess moves to 0.8: gap **0.04** (p. 49).

② **TRAIN** The guess reaches 0.9: gap **0**; the floor touches the true score.

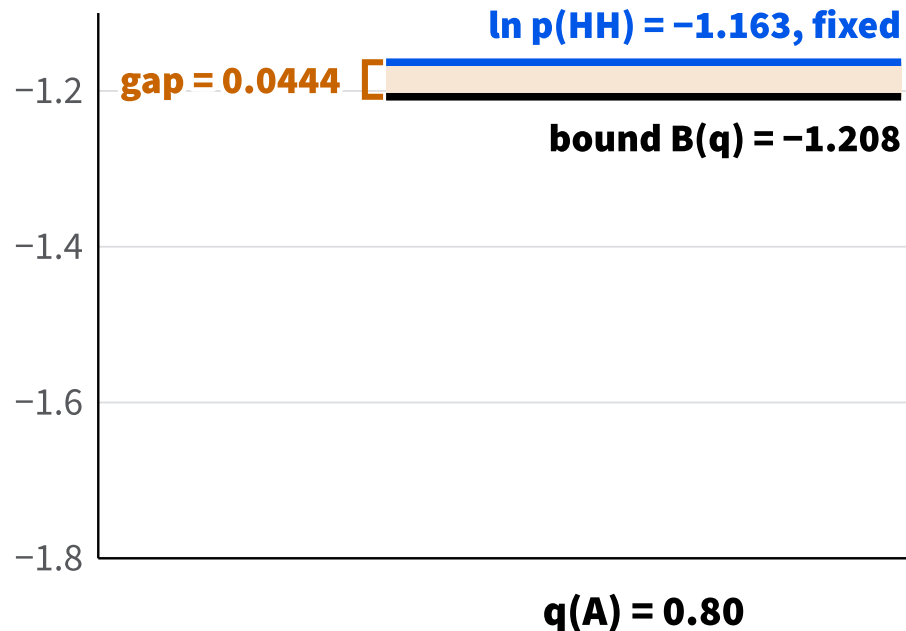
IMAGES Same game, but the encoder outputs the guess for a code in **one pass**.

Read: the encoder replaces an impossible search by a fast guess.

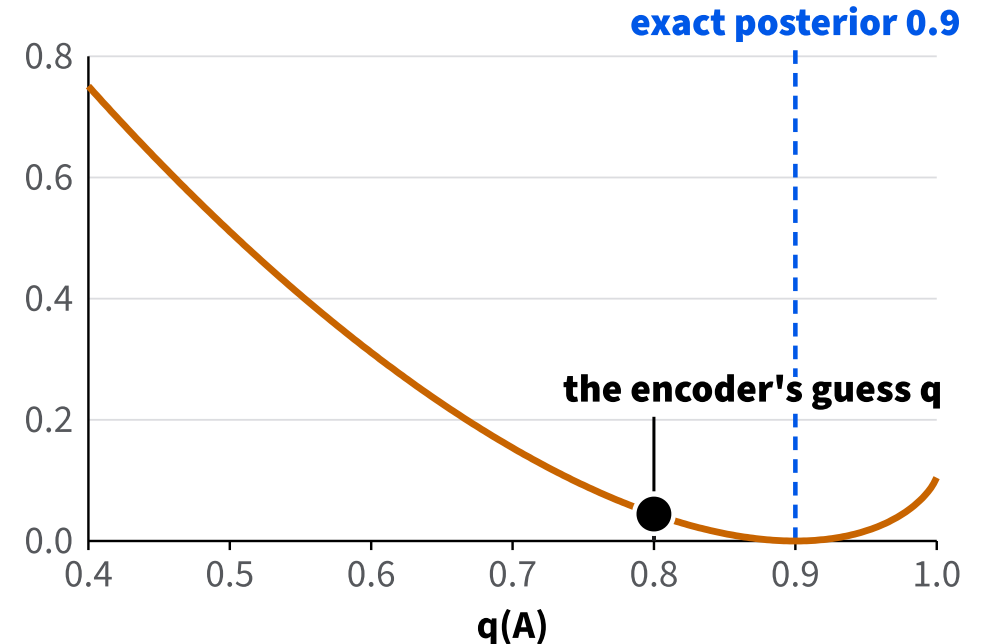
Trouble 3: the code is many numbers

Result: train the guesser, and the **gap** closes.

$$\ln p(\text{HH}) = \text{bound} + \text{gap}$$



gap = how wrong the guess is



Watch: move the guess q : the orange gap under $\ln p(\text{HH})$ shrinks to zero at the exact 0.9.

Q: Why does this scale to images? A: One pass of the encoder per image, no search.

Try: move the guess to 0.9. This is the **variational autoencoder** [Kingma and Welling, 2014].

Trouble 4 in three sentences

- BACKGROUND** A language model reads text in pieces (**tokens**): un|happi|ness. Good pieces make it faster.
- KEY IDEA** Learn the pieces from raw text: EM weighs **every cut** of every word.
- USED FOR** Tokenizers of T5 and ALBERT [Kudo, 2018].

Trouble 4: where to cut words

Trouble: nobody marks where to **cut**.

n o o n

n o o n	?
n o on	?
n oo n	?
no o n	?
no on	?

1. With the pieces n, o, no, on, oo, “noon” has **five** cuts.

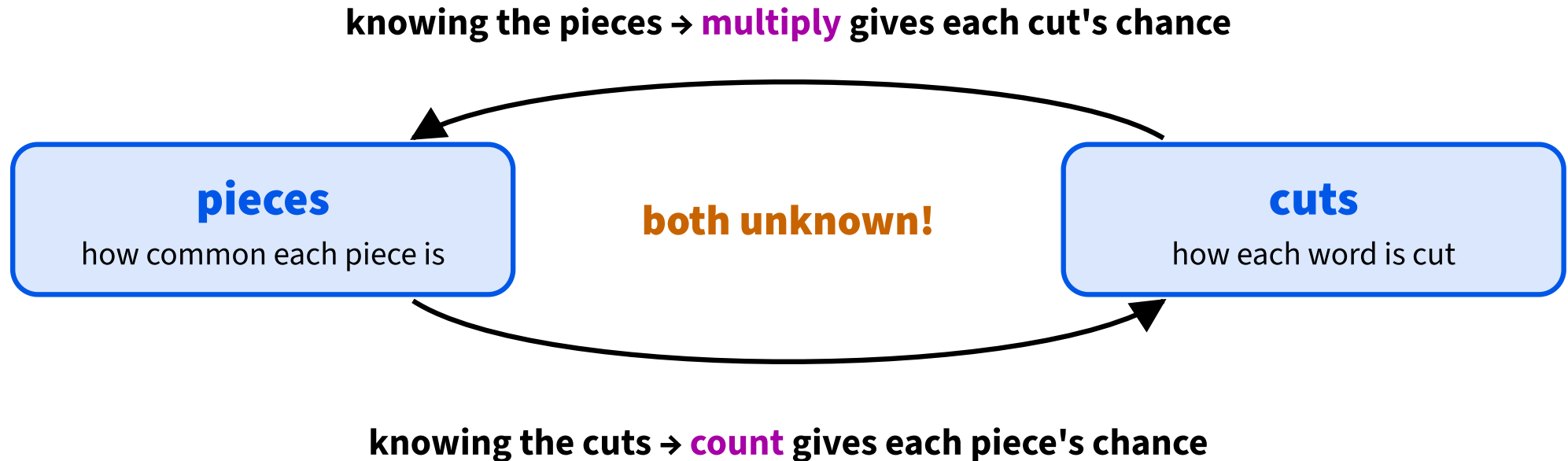
2. Which cut is right? Nobody tells us.

Q: Why not cut at every letter?

A: Four pieces where two would do: slower, and letters carry no meaning.

Trouble 4: where to cut words

Trouble: a chicken-and-egg problem again.



Read: the cut is the hidden coin; a piece's chance is θ .

Q: Where do we start? A: Make every piece equally likely, then go round the loop.

Trouble 4: where to cut words

Fix: go round the loop. One round by hand:

START Every piece equally likely: **1/5** each.

① CUTS A cut's chance = its pieces multiplied: no|on = $1/25$, three pieces = $1/125$. So no|on gets **0.61**.

② PIECES Count pieces, weighted by the cuts: **no** and **on** gain.

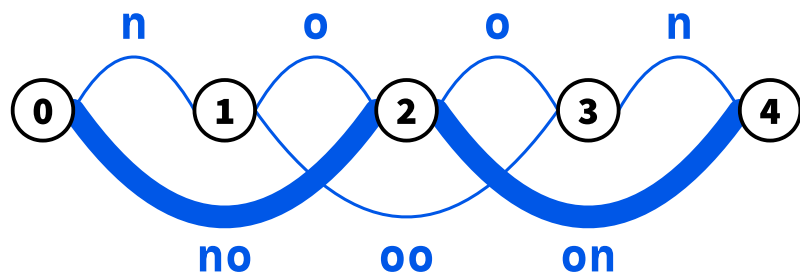
REPEAT no|on grows surer each round: **0.999** by round 4.

Read: fewer pieces means fewer small numbers multiplied, so the natural cut wins.

Trouble 4: where to cut words

Result: EM finds the natural cut **no|on**.

every path 0 → 4 is a cut of “noon”



thicker arc = likelier piece

**the cut no|on becomes
almost certain (0.999)**

E-step: share of each cut

= its chance ÷ chance of “noon”

n|o|o|n | 0.000

n|o|on | 0.001

n|oo|n | 0.000

no|o|n | 0.001

no|on | 0.999

M-step: piece probability

= expected count ÷ 2.00 pieces

n | 0.001

o | 0.001

no | 0.499

on | 0.499

oo | 0.000

Watch: round by round **no** and **on** gain, and no|on takes all the weight.

Q: Why not list every cut? A: A 20-letter word has up to 524,288; a running total adds them letter by letter.

Try: favour “oo”: EM gets stuck. This is the **Unigram tokenizer** [Kudo, 2018].

Trouble 5 in three sentences

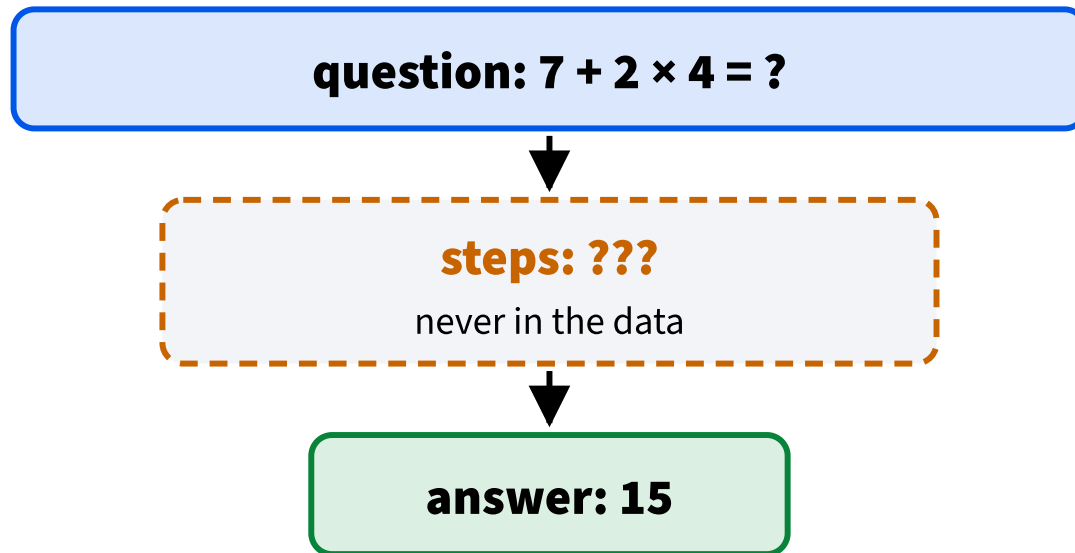
BACKGROUND To teach reasoning we have questions and right answers, but **no worked steps**.

KEY IDEA Let the model write its own steps, **keep what reaches the right answer**, learn from it.

USED FOR Teaching language models to reason [Zelikman et al., 2022].

Trouble 5: the hidden reasoning

Trouble: the worked **steps** are missing.



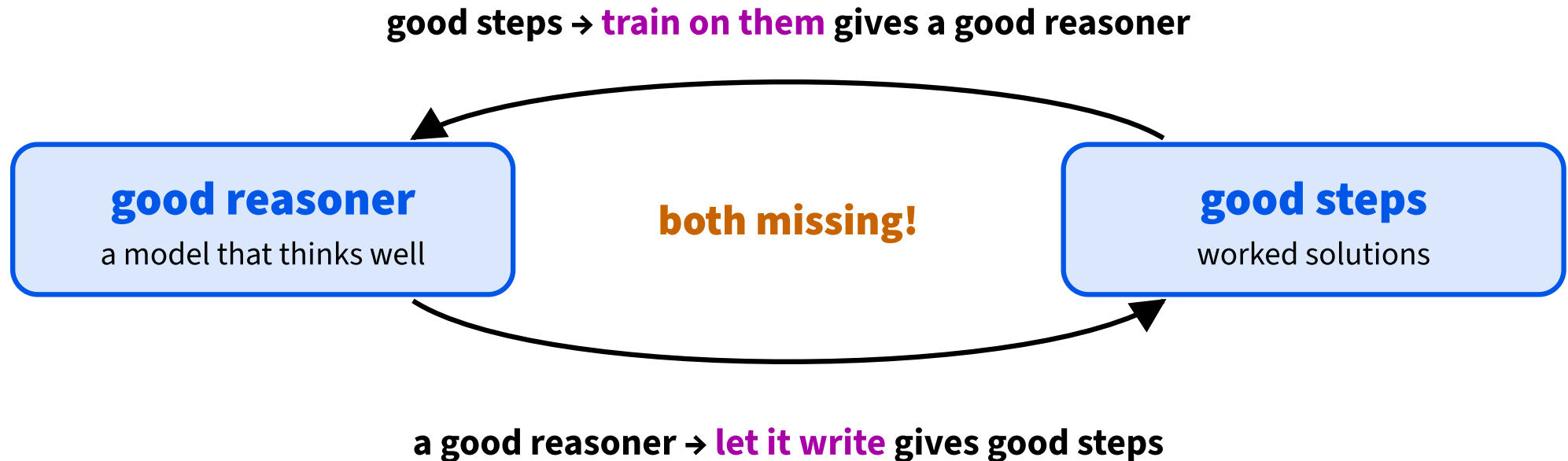
1. The data give the question and the answer.
2. The steps between are hidden, like the coin.

Q: Why not train on the answers alone?

A: The model learns to guess answers, not to reason.

Trouble 5: the hidden reasoning

Trouble: a chicken-and-egg problem again.



Read: the steps are the hidden coin; how often the model reasons carefully is θ .

Q: Where do we start? A: With the model we have, then go round the loop.

Trouble 5: the hidden reasoning

Fix: go round the loop. One round by hand:

- START** A tiny model reasons **carefully** half the time (right 3 in 4) or **hastily** (right 1 in 4).
- ① **WRITE** 8 tries: 4 careful give 3 right; 4 hasty give 1 right.
- ② **KEEP** Keep the 4 right ones: 3 careful, 1 hasty. Retrain: careful **3/4 (p. 50)**.
- REPEAT** Careful grows: $3/4 \rightarrow 0.9 \rightarrow$ nearly always.

Read: the right answer acts as a filter: it lets mostly careful steps through.

Trouble 5: the hidden reasoning

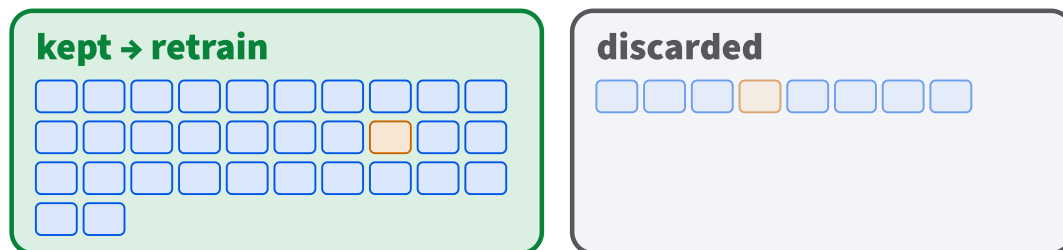
Result: careful reasoning takes over.

the model answers 40 questions

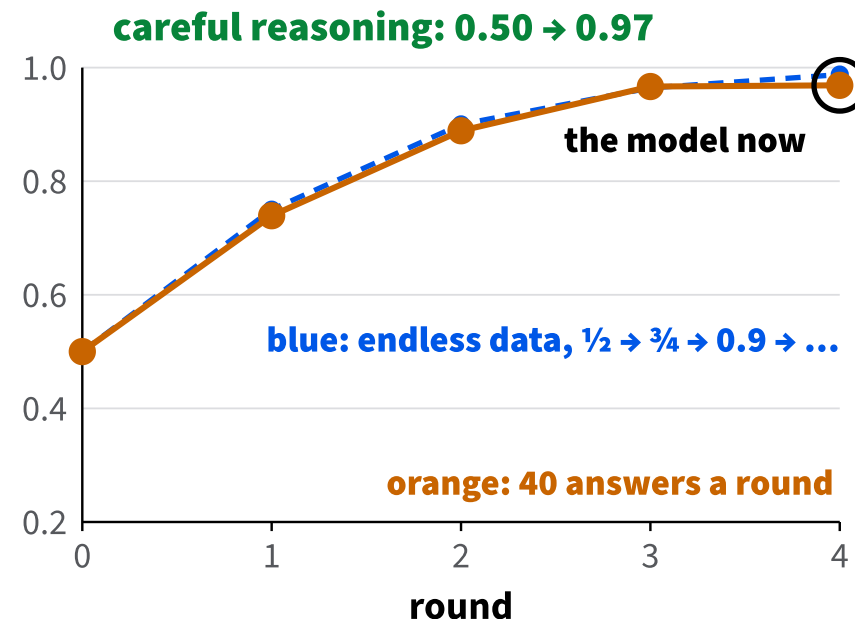
☐ careful: right $\frac{3}{4}$ ☐ hasty: right $\frac{1}{4}$

kept 32: 31 careful, 1 hasty

retrain: new $s = 31/32 = 0.969$



s = share of careful reasoning



Watch: each round the model writes 40 solutions, keeps the right ones and retrains: careful rises from 0.50 to 0.97.

Q: Why not keep every solution? A: Then nothing changes: careful stays at 1/2.

Try: keep all. This is STaR, a **Monte Carlo EM** [Zelikman et al., 2022; Phan et al., 2023].

Outline

Learning what we never see (Lecture 5)

Troubles 1–2: cleaning and searching data

Troubles 3–5: inside image and language models

► **Conclusion and your turn**

Finally: all five fixes, and your turn.

Conclusion

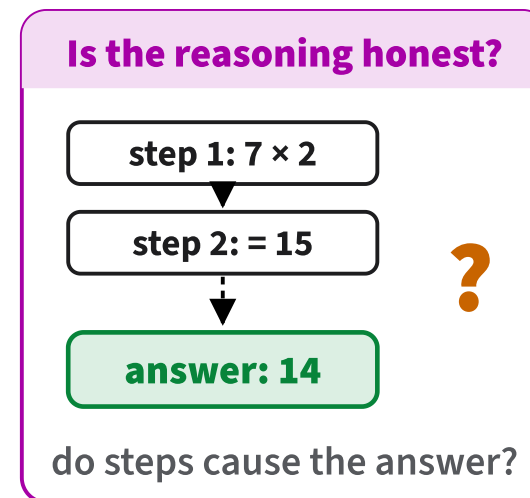
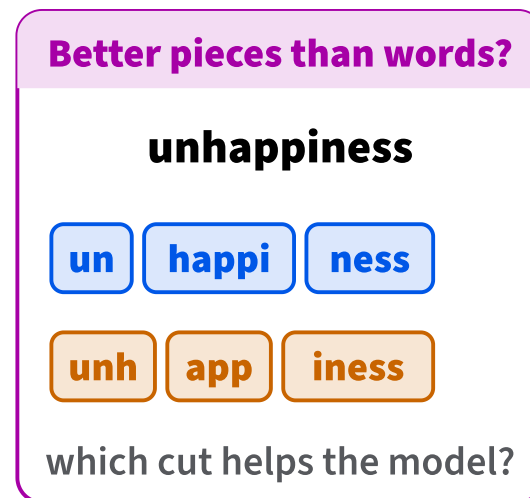
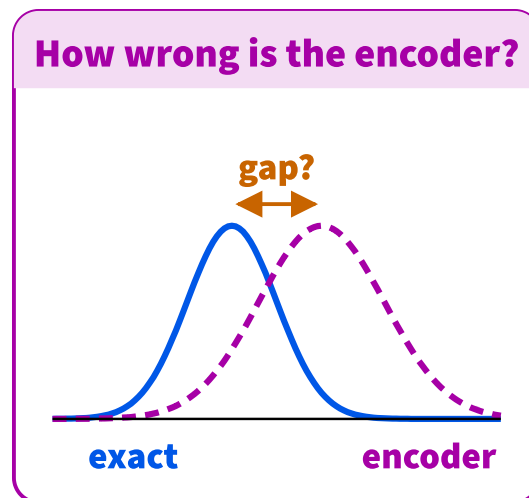
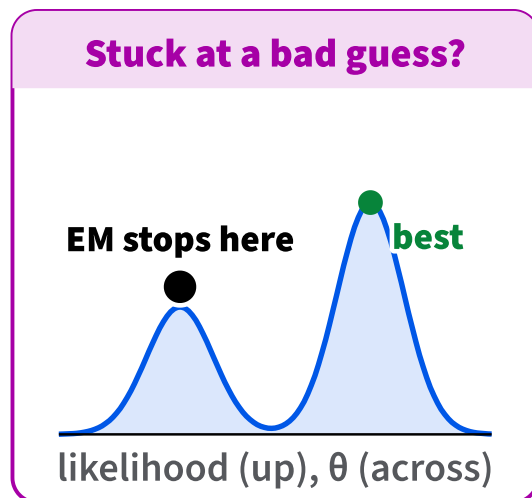
One loop, five small changes.

	Trouble	Fix	Used for
1	people disagree on labels	trust who agrees	clean training labels
2	too much data	search one cell	search for chatbots
3	the hidden part is many numbers	a fast guesser	image models
4	nobody marks where to cut words	weigh every cut	tokenizers
5	the reasoning is never written	keep what works	models that reason

Read: every fix keeps **guess**, then **count**.

Conclusion

- ▶ We can **learn what we never see**: guess, then count (expectation–maximization).
- ▶ Four **open problems**:



Project seeds: Jin et al. (2016), Cremer et al. (2018), Pagnoni et al. (2025), Turpin et al. (2023).

Your turn

Predict, then try.

In these slides

- ▶ another start (p. 5): still 0.789?
- ▶ a careless labeller (p. 14)
- ▶ more cells (p. 21)
- ▶ the encoder's guess (p. 27)
- ▶ “oo” first (p. 32); keep all (p. 37)

After class: lab5_explore.py

1. Two coins from $\theta = (0.6, 0.6)$: where to?
2. Eight photos: how much is person 3 trusted?
3. k-means: how fast, how many misses?
4. A tiny reasoner: keep right, or keep all?

Course site → Week 5 → Lab files. One or two lines each.

Tip: write your guess first; surprises teach.

Selected References I

Start here: Dempster, Laird and Rubin (1977), where EM got its name.



Cremer, C., Li, X., and Duvenaud, D. (2018).

Inference suboptimality in variational autoencoders.

In Proceedings of the 35th International Conference on Machine Learning, PMLR 80:1078–1086.



Dawid, A. P. and Skene, A. M. (1979).

Maximum likelihood estimation of observer error-rates using the EM algorithm.

Journal of the Royal Statistical Society, Series C (Applied Statistics), 28(1):20–28.



Dempster, A. P., Laird, N. M., and Rubin, D. B. (1977).

Maximum likelihood from incomplete data via the EM algorithm.

Journal of the Royal Statistical Society, Series B (Methodological), 39(1):1–38.



Jégou, H., Douze, M., and Schmid, C. (2011).

Product quantization for nearest neighbor search.

IEEE Transactions on Pattern Analysis and Machine Intelligence, 33(1):117–128.



Jin, C., Zhang, Y., Balakrishnan, S., Wainwright, M. J., and Jordan, M. I. (2016).

Local maxima in the likelihood of Gaussian mixture models: structural results and algorithmic consequences.

In Advances in Neural Information Processing Systems 29, pages 4116–4124.

Selected References II

Entry points: Kingma and Welling (2014); Kudo (2018).



Kingma, D. P. and Welling, M. (2014).

Auto-encoding variational Bayes.

In 2nd International Conference on Learning Representations; arXiv:1312.6114.



Kudo, T. (2018).

Subword regularization: improving neural network translation models with multiple subword candidates.

In Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics, pages 66–75.



Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., and Kiela, D. (2020).

Retrieval-augmented generation for knowledge-intensive natural language processing tasks.

In Advances in Neural Information Processing Systems 33, pages 9459–9474.



Lloyd, S. P. (1982).

Least squares quantization in pulse-code modulation.

IEEE Transactions on Information Theory, 28(2):129–137.

Selected References III

A short read: Zelikman et al. (2022), a model teaching itself to reason.



Pagnoni, A., Pasunuru, R., Rodriguez, P., Nguyen, J., Muller, B., Li, M., Zhou, C., Yu, L., Weston, J., Zettlemoyer, L., Ghosh, G., Lewis, M., Holtzman, A., and Iyer, S. (2025).

Byte latent transformer: patches scale better than tokens.

In Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 9238–9258.



Phan, D., Hoffman, M. D., Dohan, D., Douglas, S., Le, T. A., Parisi, A., Sountsov, P., Sutton, C., Vikram, S., and Sauros, R. A. (2023).

Training chain-of-thought via latent-variable inference.

In Advances in Neural Information Processing Systems 36.



Turpin, M., Michael, J., Perez, E., and Bowman, S. R. (2023).

Language models don't always say what they think: unfaithful explanations in chain-of-thought prompting.

In Advances in Neural Information Processing Systems 36.



Zelikman, E., Wu, Y., Mu, J., and Goodman, N. D. (2022).

STaR: bootstrapping reasoning with reasoning.

In Advances in Neural Information Processing Systems 35, pages 15476–15488.

Thank you!

Next: Lecture 6, when even the guess must be approximated

EM in 11 lines of Python: see the backup slides.

Backup: one round of Lecture 5

One round from $\theta = (\frac{3}{4}, \frac{1}{4})$, coins picked $\frac{1}{2}$ each.

$$p(A \mid \text{HH}) = \frac{9/32}{9/32 + 1/32} = \frac{9}{9 + 1} = 0.9$$

$$p(A \mid \text{HT}) = \frac{3}{3 + 3} = 0.5, \quad p(A \mid \text{TT}) = \frac{1}{1 + 9} = 0.1$$

$$\text{heads for A} = 0.9 \cdot 2 + 0.5 \cdot 1 + 0.1 \cdot 0 = 2.3$$

$$\text{flips for A} = (0.9 + 0.5 + 0.1) \cdot 2 = 3$$

$$\theta_A = \frac{2.3}{3} = 0.767, \quad \theta_B = \frac{3 - 2.3}{6 - 3} = 0.233$$

Notice: heads $\div$ flips, each record weighted by its soft label.

E-step: $\frac{1}{2} \cdot \frac{3}{4} \cdot \frac{3}{4} = 9/32$ against $\frac{1}{2} \cdot \frac{1}{4} \cdot \frac{1}{4}$

the same rule

soft label $\times$ heads

soft label $\times$ flips

M-step: count $\div$ total

Backup: one round of Lecture 5

EM in 11 lines of Python.

```
records = ["HH", "HT", "TT"]
a, b = 0.75, 0.25                                # starting guess: heads chances of coins A and B
for r in range(1, 31):
    heads = flips = 0.0
    for x in records:
        h, t = x.count("H"), x.count("T")
        pa, pb = a**h * (1-a)**t, b**h * (1-b)**t  # chance of the record from each coin
        s = pa / (pa + pb)                          # E-step: soft label for coin A
        heads += s * h; flips += s * (h + t)        # count with the soft label
    a, b = heads / flips, (3 - heads) / (6 - flips)  # M-step: expected heads / expected flips
    if r in (1, 2, 30): print(r, round(a, 3), round(b, 3))
1 0.767 0.233
2 0.777 0.223
30 0.789 0.211
```

Try: start from $a = b = 0.5$. What happens?

Backup: Trouble 1

One round on the six photos: every accuracy starts at 0.8, cat and dog $\frac{1}{2}$ each.

$$p(\text{cat} \mid \text{cat}, \text{cat}, \text{dog}) = \frac{0.064}{0.064 + 0.016} = 0.8$$

$$p(\text{dog} \mid \text{dog}, \text{dog}, \text{dog}) = \frac{0.256}{0.256 + 0.004} = 0.9846$$

$$a_1 = a_2 = \frac{0.8 + 0.8 + 4 \cdot 0.9846}{6} = 0.923$$

$$a_3 = \frac{0.2 + 0.2 + 4 \cdot 0.9846}{6} = 0.723$$

Notice: accuracy is counted like heads $\div$ flips, with “right” as heads.

E-step, photos 1 and 4

E-step, photos 2, 3, 5, 6: all agree

expected right $\div$ photos

person 3 is outvoted on photos 1 and 4

Backup: Trouble 3

The gap for the guess 0.8 on HH (exact: 0.9).

Lemma 1: $\ln p(x) = \text{bound} + \text{gap}$; the gap is the Kullback–Leibler divergence.

$$\begin{aligned}\text{gap} &= 0.8 \cdot \ln \frac{0.8}{0.9} + 0.2 \cdot \ln \frac{0.2}{0.1} \\ &= 0.8 \cdot (-0.1178) + 0.2 \cdot 0.6931 \\ &= -0.0942 + 0.1386 = 0.0444\end{aligned}$$

guess against exact, for A and for B

the two logarithms

the bound is 0.0444 too low

Notice: at the exact guess 0.9 both logs are 0: no gap.

Backup: Trouble 5

The tiny reasoner: careful = coin A, hasty = coin B, “right” = heads.

Careful: chance $\pi = \frac{1}{2}$, right $\frac{3}{4}$. **Hasty:** right $\frac{1}{4}$.

$$p(\text{careful and right}) = \frac{1}{2} \cdot \frac{3}{4} = \frac{3}{8}, \quad p(\text{hasty and right}) = \frac{1}{2} \cdot \frac{1}{4} = \frac{1}{8}$$

$$\text{new } \pi = \frac{3/8}{3/8 + 1/8} = 0.75$$

$$\text{keep all: } \pi = \frac{1/2}{1/2 + 1/2} = 0.5$$

sample, then check

share of careful among kept

nothing is learned

Notice: keeping = E-step (careful given right); retraining = M-step.